AULA 3 – UM PROCESSADOR MULTICORE MODERNO.

Prof. Rodrigo Gonçalves

www.linkedin.com/in/rodrigouzz



Stanford
University



Centro Universitário
A Teoria e a Prática Juntas

A Modern Multi-Core Processor?



Evolução dos processadores multi - core.

Prof. Rodrigo Gonçalves IESB

Aula passada, vimos memória cache como recurso para diminuir a latência. Hoje, vamos utilizar o código para evoluir o raciocínio.



Veja o código que calcula o seno de x pela expansão em série de Taylor.

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

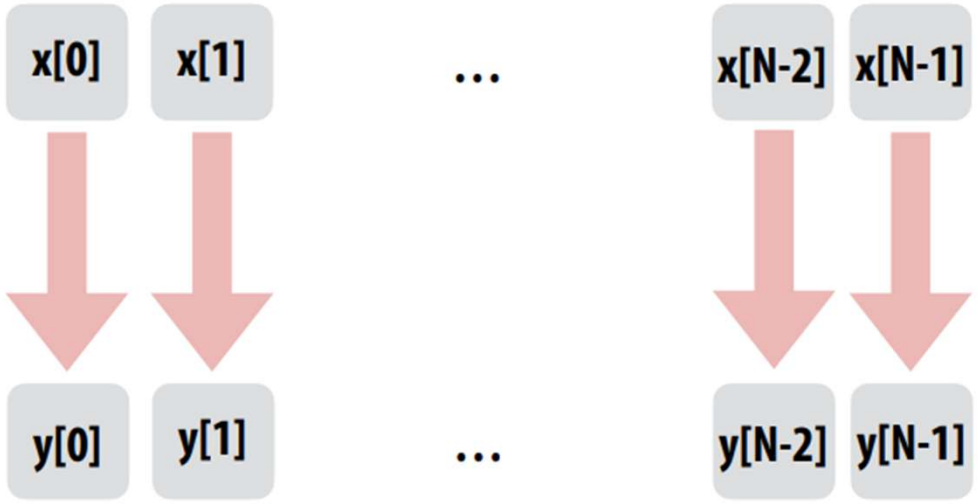
      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Compute $\sin(x)$ using Taylor expansion:

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$$

for each element of an array of N floating-point numbers



Veja o código que calcula o seno de x pela expansão em série de Taylor.

→ **Contexto: o que é a série de Taylor?**

É uma forma de aproximar funções como (sin (x), cos(x) exp(x) através de um somatório finito de polinômios.

Princípio: se é conhecida as derivadas de uma função em torno x(0)

Mas, não conhecemos os valores ao redor de x(0), utilizamos a **Expansão em série de Taylor** para estimar valores aproximados em **Em torno de x(0), por exemplo, ou outro pronto qualquer.**

Prof. Rodrigo Gonçalves IESB

Veja o código que calcula o seno de x pela expansão em série de Taylor.

→ **Contexto: o que é a série de Taylor?**

É uma forma de aproximar funções como (sin (x), cos(x) exp(x) através de um somatório finito de polinômios.

Princípio: Se conhecemos as derivadas de uma função em torno de um ponto (geralmente x=0), podemos usar essas informações para construir uma aproximação polinomial que estima os valores da função próximos a esse ponto.

Quando não sabemos calcular diretamente o valor exata da função: sin(x), cos(x), por exemplo, podemos utilizar a série de Taylor para obter valores **aproximados.**

Prof. Rodrigo Gonçalves IESB

→ **Não é preciso entender o cálculo, mas sim o que o código faz!**

Vamos ver o código novamente e entende-lo.

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

void sinx(int N, int terms, float* x, float* y)

N: tamanho do vetor de entrada,

terms: número de termos da série de Taylor que serão usados.

x: vetor de entrada com os valores de onde queremos calcular sin(x).

y: vetor de saída que são os resultados de sin(x) para cada x[i].

O que importa aqui é a entrada de um array e a saída desse array. Não se importe como é calculado.

→ **Entrada array x[i] e saída array y[i]**

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

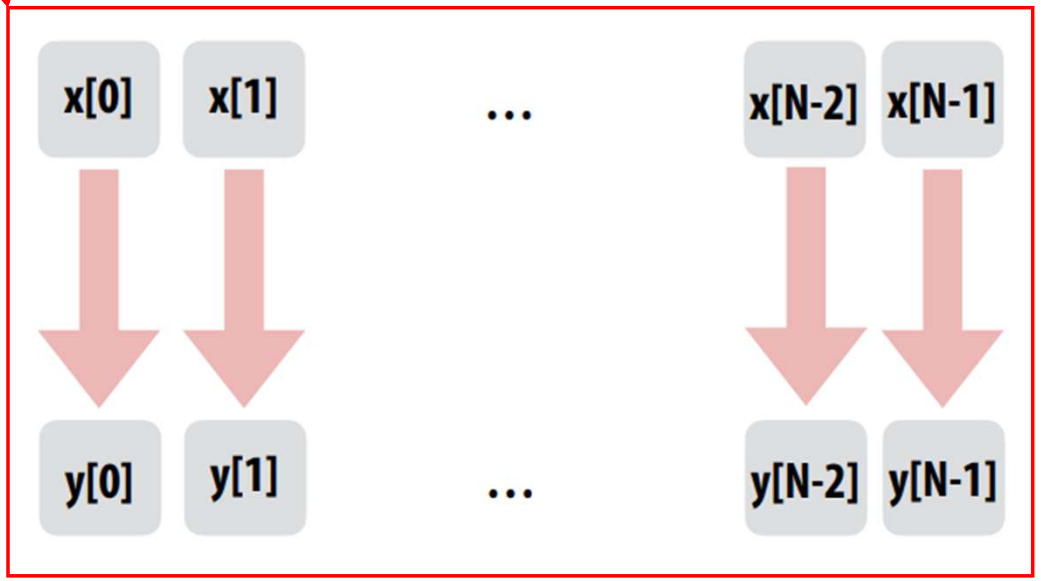
      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

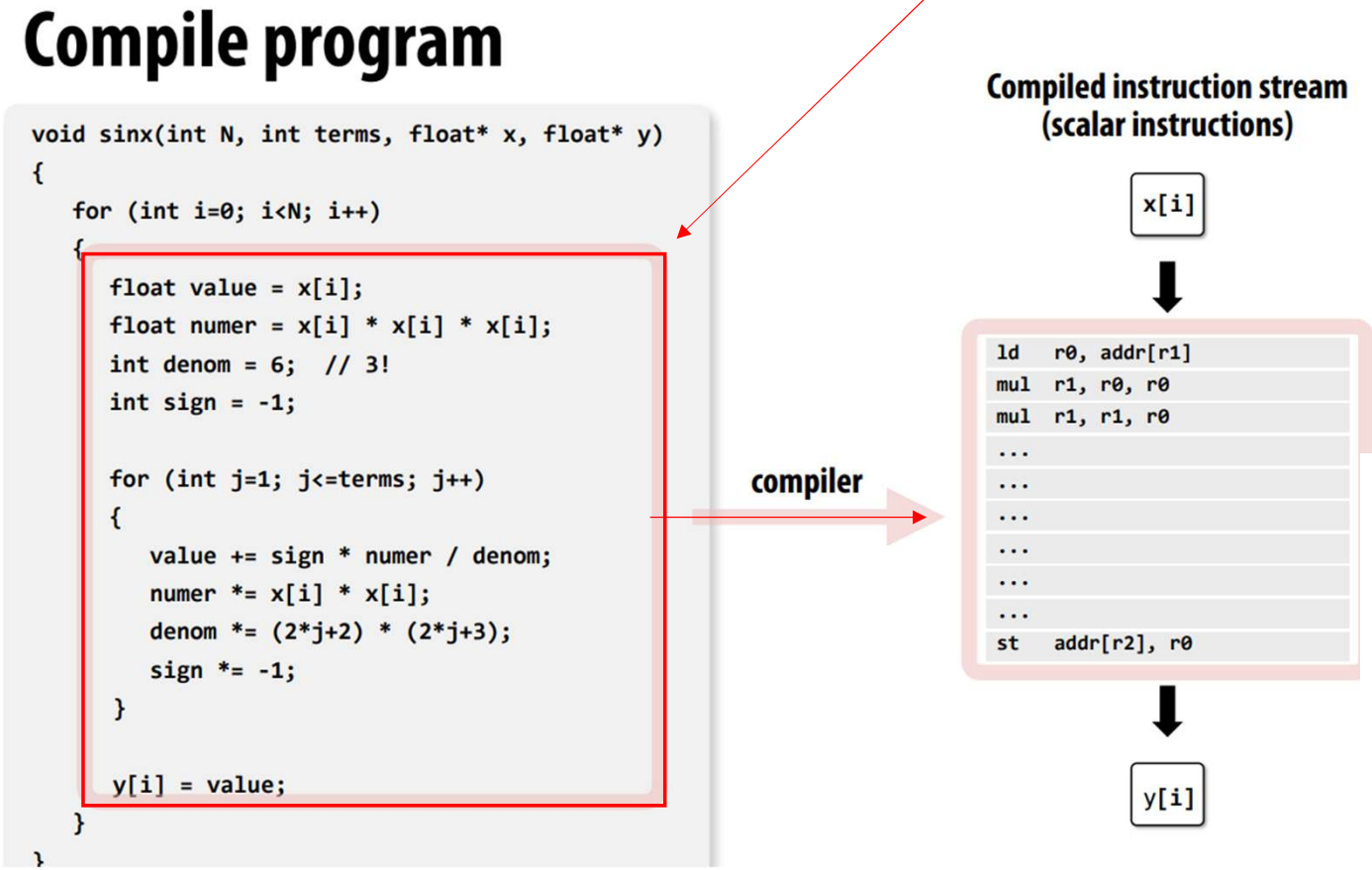
Compute $\sin(x)$ using Taylor expansion:

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$$

for each element of an array of N floating-point numbers



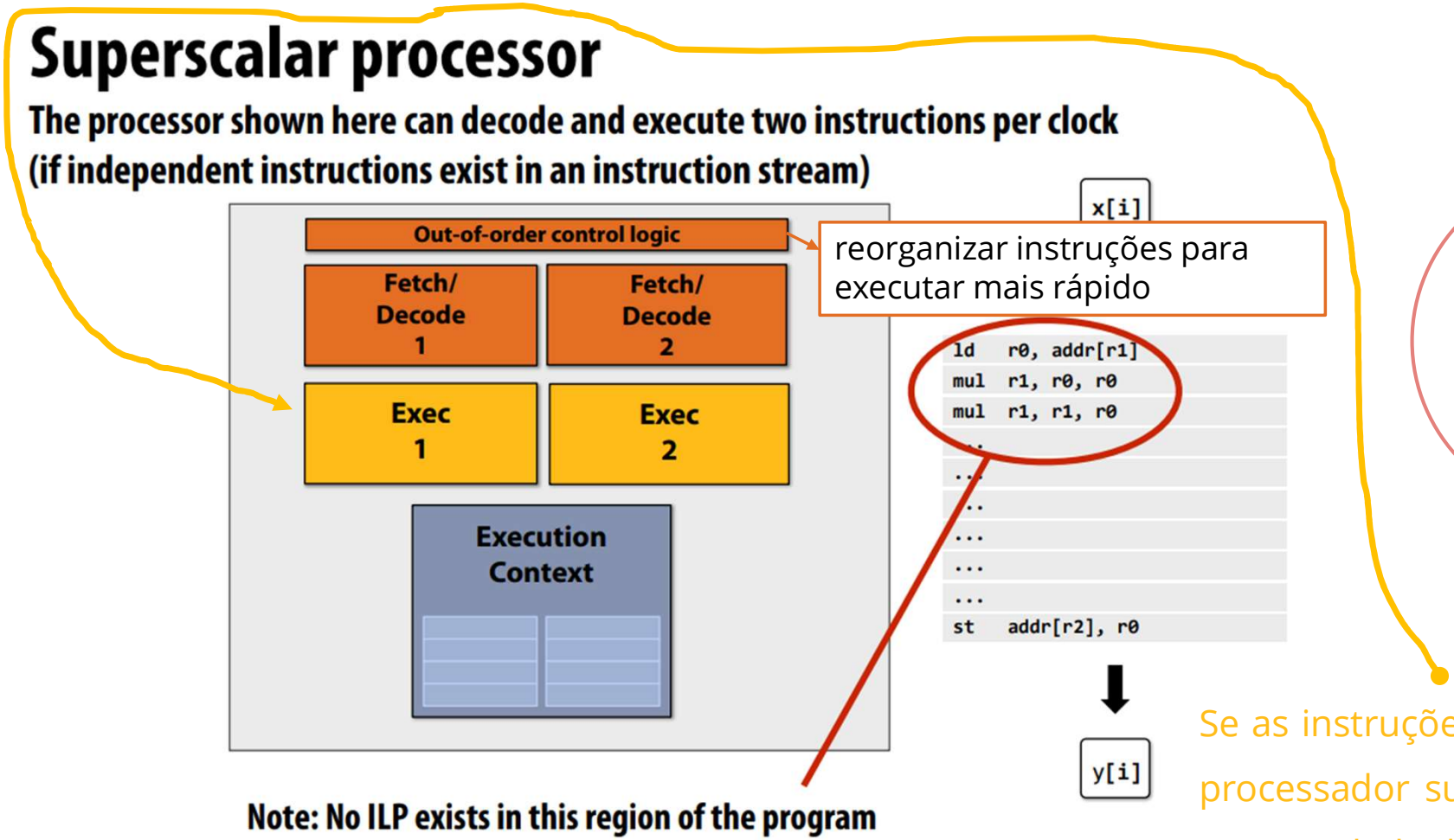
→ **Se** compilarmos esse programa com GCC ou Clang, temos uma sequência de instruções, certo? Vamos focar na sequência dessas instruções. Mais precisamente as instruções dentro do for.



→ Ou seja, uma vez tendo carregado os dados x[i], vamos projetar a saída y[i].

→ Vamos executar essas instruções... Próximo slide.

→ Lembrando que esse processador, executa uma instrução por clock. Mas como sabemos, o processador superescalar, e olha nas instruções e pensa, dada essas instruções, o que eu posso fazer em paralelo? Pra isso ele reorganiza as instruções.



→ Essas instruções são todas dependentes

Se as instruções são todas dependentes o processador superescalar utilizará apenas uma unidade de execução

Prof. Rodrigo Gonçalves IESB

→ **No entanto, temos um problema.**

se considerarmos o laço for, sempre teremos dependência , pois temos o i++.

O que os arquitetos de hardware pensaram?

Vamos deixar que os programadores nos digam o que é para executar em paralelo, nós não precisamos nos preocupar taaaanto com isso. Veja a ideia anterior, com uma arquitetura melhorada (Keyvon).

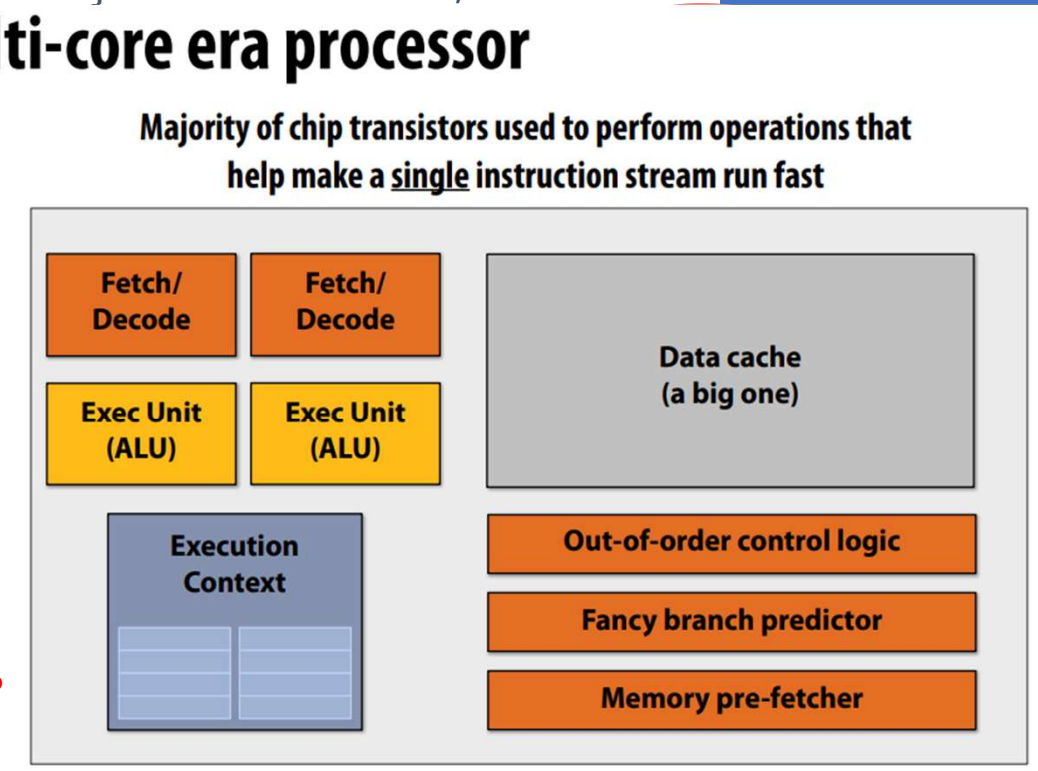
Nos anos 90 e inicio de 2000, a estratégia para acelerar os programas, era colocar cada vez mais inteligência em um único core.

→ out-of-order: reorganizar as intruções

→ Branch predictor (“prevêr” qual caminho um if vai seguir, por exemplo.

→ Caches maiores.

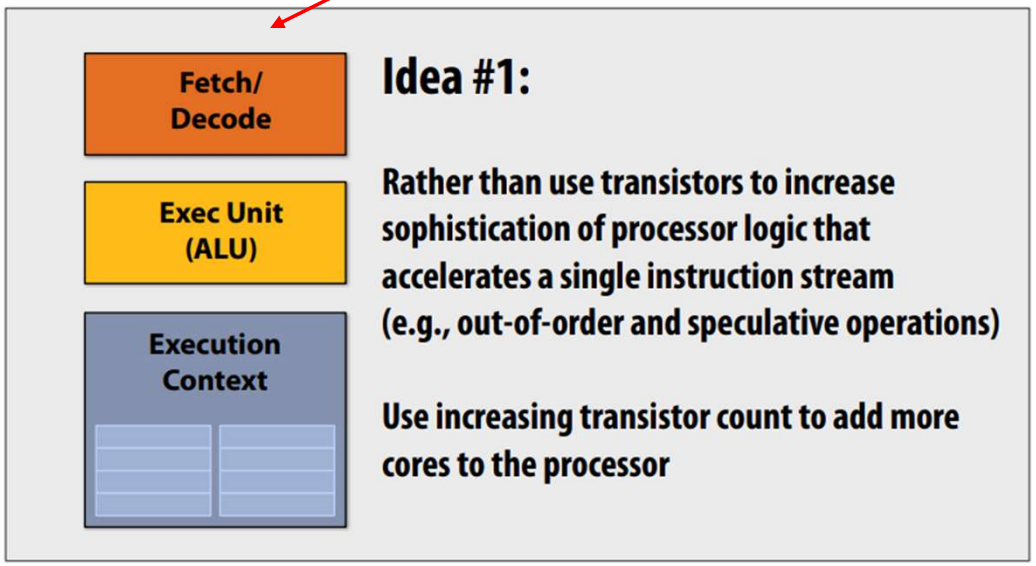
TUDO isso = + transistores!, logo como melhorar isso?



→ Pensaram... Ideia #1.

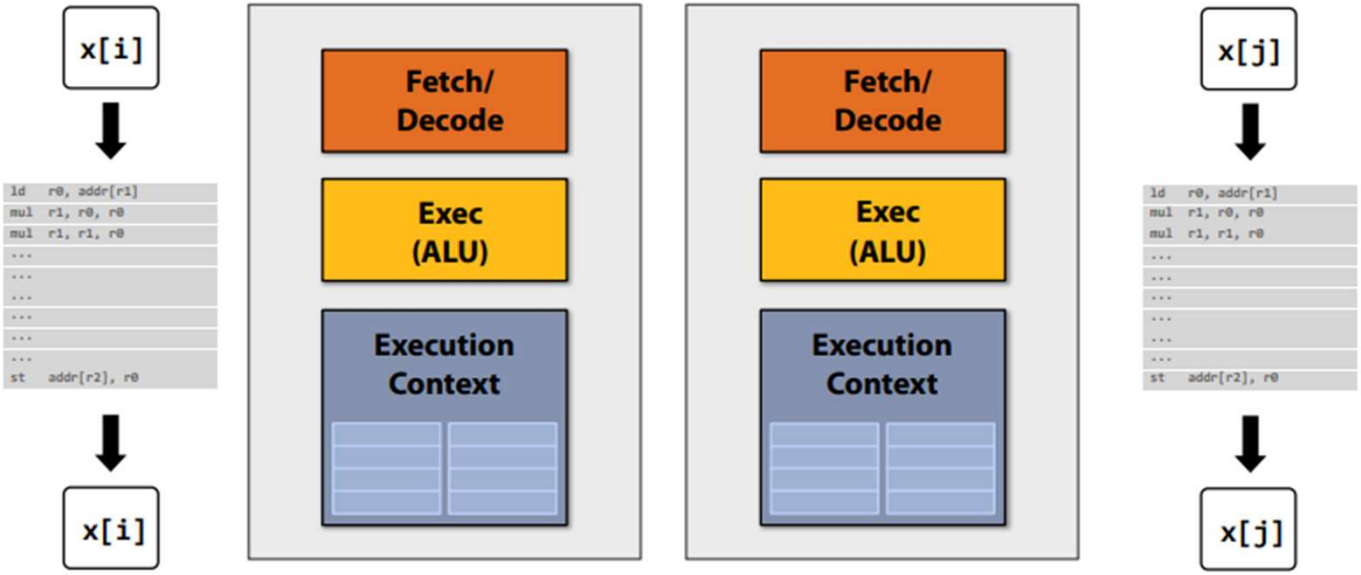
Invés de colocar mais inteligência nos processadores, vamos adicionar mais cores (núclos) a eles. **Ou seja, vamos replicar esse tipo de processador**

Multi-core era processor



Exemplo, próximo slide...

→Two cores: calcula dois elementos do array ao mesmo tempo.



Essa arquitetura produz um **resultado 25% mais lento** do que um super- core ou processador superescalar com um monte de inteligência. No entanto, **ainda ganha 1,5x mais desempenho.**

Desempenho core antigo = velocidade 1.0

Novo core simples = velocidade 0,75.

Dois cores paralelos , temos:

desempenho_total = 2 x 0,75 = 1,5 (potencial para speedup).

MAS.....

→Mas...., nosso programa exemplo não expressa paralelismo... Ele é escrito serialmente.

Isso significa que o compilador gera apenas um fluxo de de instruções, que será executado em uma única thread.

O processador, mesmo tendo 2 cores disponíveis, não tem como automaticamente dividir esse trabalho entre eles, porque o programa não foi escrito para expor paralelismo

But our program expresses no parallelism

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
```

This C program will compile to an instruction stream that runs as one thread on one processor core.

então, usando 2 cores, ele ficará 25% mais lento, do que utilizando um único processador mais inteligente.

Logo, ter hardware paralelo, não basta!

O software precisa expor paralelismo!

Continua ....

→... Continuação.

Se o programa é escrito de forma sequencial ou em série, não importa quantos cores o processador tenha, ele não consegue acelerar.

→ **Para isso, temos que utilizar ferramentas como: Threads, OpenMP, Cuda, para dizer explicitamente:**

→ **"essas iterações são independentes, rode em paralelo."**



O hardware não advinha paralelismo, nós, programadores, precisamos escrever o código de forma que o compilador ou o sistema consiga paralelizar

Então, temos que mudar o nosso código.

Então, mudamos o código..

Adicionei duas Threads. Mas, o que são Threads?

```
typedef struct {
   int N;
   int terms;
   float* x;
   float* y;
} my_args;

void my_thread_func(my_args* args)
{
   sinx(args->N, args->terms, args->x, args->y); // do work
}
```

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Stanford CS149



Threads = é a menor unidade de execução que o sistema operacional consegue agendar para rodar no núcleo disponível do processador.

→ Uma thread para o programador, é como se fosse uma linha de execução dentro de um programa.

→ Um programa, pode ter várias Threads.

→ 2 cores, pode executar 2 Threads.



Vamos quebrar em partes esse código para entendê-lo.

Prof. Rodrigo Gonçalves IESB

```
typedef struct {
   int N;
   int terms;
   float* x;
   float* y;
} my_args;

void my_thread_func(my_args* args)
{
   sinx(args->N, args->terms, args->x, args->y); // do work
}
```

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Parte 1:

Prof. Rodrigo Gonçalves IESB

```
typedef struct {
   int N;
   int terms;
   float* x;
   float* y;
} my_args;

void my_thread_func(my_args* args)
{
   sinx(args->N, args->terms, args->x, args->y); // do work
}
```

typedef struct{...} my_args; criar uma estrutura com parâmetros.

int N = número de elementos subtamanho do vetor
int terms = número de termos da série de Taylor.
float* x = ponteiro para os dados de entrada
Float *y = ponteiro para os dados de saída.

(my_args* args) = args recebe a estrutura criada my_args, como um ponteiro

Passa os valores que calcula a sinx.



Parte 2:

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```

instancia um objeto std::thread vazio (ainda não joinable)

chama a estrutura que criamos (my_args) passando para args.

tamanho do pedaço que a thread vai processar, dividido por 2. Objetivo, dividir o array em 2

termos da série de Taylor.

Vetor de entrada começa em x[0]
Vetor de saída começa em y[0]



Se N = 6 , ou e seja (PAR)

args.N = N/2 = 6/2 = 3. Então,
→**Thread secundária** processa os 3 primeiros elementos, índices[0, 1, 2]

→**Thread principal**: processa os N - args.N = 6 – 3 = 3 restantes, índices [3, 4, 5] (slice).

Resultado: os 6 elementos foram divididos igualmente 3 e 3.

Parte 2:

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```

instancia um objeto std::thread vazio (ainda não joinable)

chama a estrutura que criamos (my_args) passando para args.

tamanho do pedaço que a thread vai processar, dividido por 2. Objetivo, dividir o array em 2

termos da série de Taylor.

Vetor de entrada começa em x[0]
Vetor de saída começa em y[0]



Se N = 5 , ou e seja (IMPAR)

args.N = N/2 = 5/2 = 2. LEMBRAR, em C a divisão de inteiros há o descarte da parte decimal,
→**Thread secundária** processa os 2 primeiros elementos, índices[0, 1]

→**Thread principal**: processa os N - args.N = 5 – 2 = 3 restantes, índices [2, 3, 4] (slice).

Resultado: os 5 elementos foram divididos igualmente em 2 e 3.

Parte 3:

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```



→ my_thread = std::thread(my_thread_func, &args); → lança uma thread (reserva, um núcleo de execução) de uma parte do array (vetor), em seguida....

→ sinx(N –args.N, terms, x + args.N, y + args.N) → lança outra thread, reserva outro núcleo de execução em paralelo com a segunda parte do vetor.

→ Join() = bloqueia a thread principal, até que a thread my_thread termina.

Parte 4:

Porque Join()?

→ Com o Join garantimos que:

1. Toda a primeira metade do cálculo foi calculda.
2. É seguro prosseguir porque não há mais threads acessando a memória local, ou seja, acontece um (lock na memória).
3. Todos os resultados já estão prontos na saída do array [y]

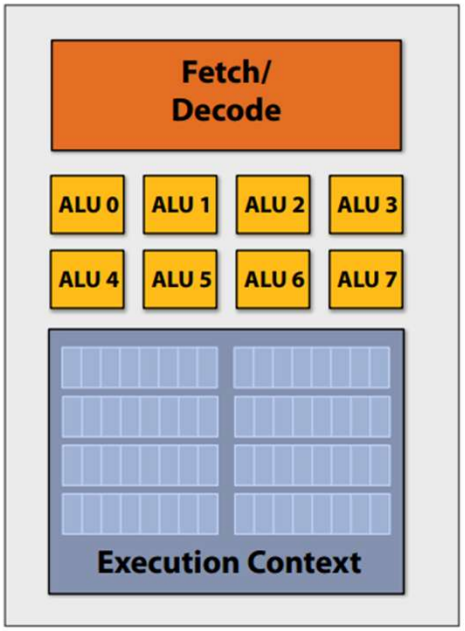


Outra forma de paralelizar uma parte da execução de programas.

Adicionando ao processador ALUs(Arithmetic Logic Unit) dentro do core, para aumentar a capacidade de computação.
Temos 8 ALUs.

→ ALU é o responsável pera aritmética dentro do core: faz contas: add, mul, comparações.

Add execution units (ALUs) to increase compute capability



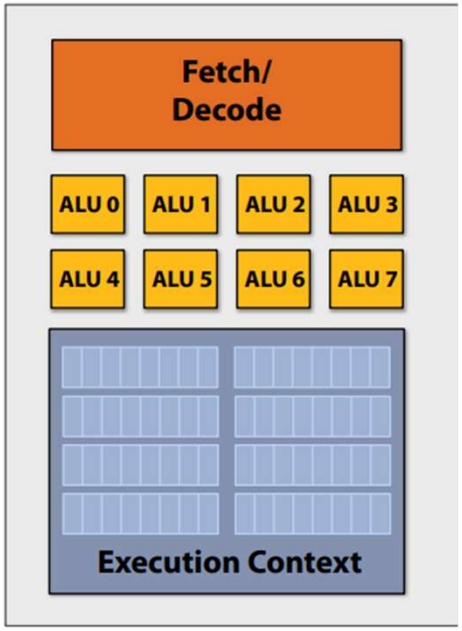
Idea #2:
Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD processing
Single instruction, multiple data

Same instruction broadcast to all ALUs
This operation is executed in parallel on all ALUs

Uma instrução, como uma soma por exemplo, para 8 valores diferentes.
Agora, temos um vetor com 8 posições.

Como funciona o vetor de ALUs?



Vamos supor que iremos multiplicar cada elemento de x por 2, em um vetor de 8 elementos , que está em alguma parte do meu código.

```
float x[8] = {1, 2, 3, 4, 5, 6, 7, 8};
float y[8];
```

A execução dessa multiplicação sem SIMD, ou seja com uma ALU. seria uma instrução por vez.

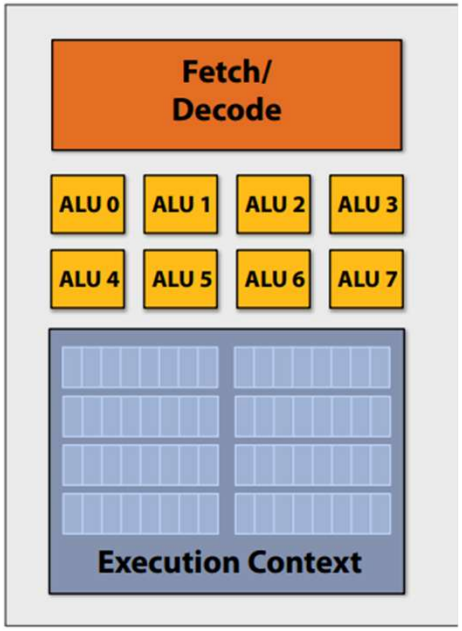
```
y[0] = x[0] * 2
y[1] = x[1] * 2
...
y[7] = x[7] * 2
```

Com SIM 8 ALUs: com uma instrução eu consigo fazer essa multiplicação em paralelo, pegando os valores dos registradores.

```
y[0..7] = x[0..7] * 2
```

Como funciona o vetor de ALUs?

Prof. Rodrigo Gonçalves IESB



Instrução: MUL 2

ALU0	ALU1	ALU2	ALU3	ALU4	ALU5	ALU6	ALU7
x[0]	x[1]	x[2]	x[3]	x[4]	x[5]	x[6]	x[7]
↓	↓	↓	↓	↓	↓	↓	↓
y[0]	y[1]	y[2]	y[3]	y[4]	y[5]	y[6]	y[7]

IESB
Centro Universitário
A Teoria e a Prática Juntas

Exercícios

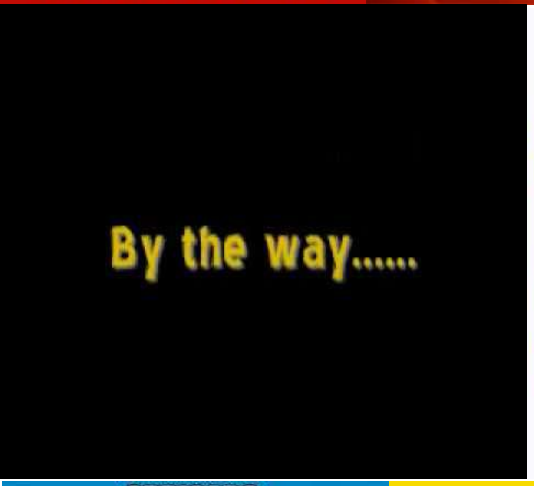



THANKS
FOLKS !!
AND
HOPE YOU
ENJOYED


That's all Folks!


Thank You!


Obrigado
Thank You
Danke
Merci
Grazie
Spasibo
Arigato
Kiitos
Chokrane
Maake
Dankon
Matondo
Mochchakkeram
Welalin
Kia Ora
Vinaka
Mamana
Terma Kasih
Multumesc
Juspaxar
Danke
Cam on ban
Nirringrazzjak
Obrigado
Asante
Dank Je
Salamat
Matur Nuwun
Raibh Maith Agat
Ua Tsaug Rau Koj

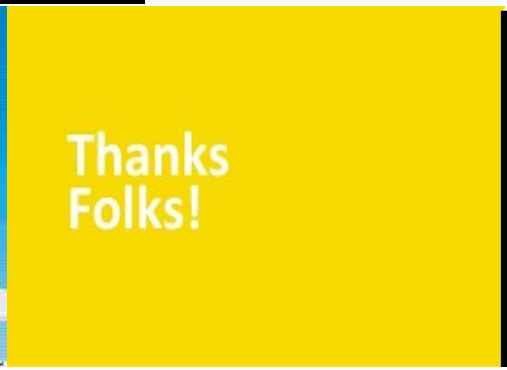

By the way.....


OBRIGADO!


GRACIAS
THANK
YOU
ARIGATO
SHUKURIA
BOLZÏN
MERCI
BÏYAN
SHUKRIA
DANKSCHEEN
TASHAKKUR ATU
SUKSAMA
GRAZIE
MEHRBANI
JUSPATAR


OBRIGADO


OBRIGADO
PELA ATENÇÃO


Thanks
Folks!


Thank You!


Thank You !